Звіт про виконання завдання № 5.3

Функції, що містять розгалуження та цикли з рекурентними співвідношеннями

Варіант 7

Виконав: Джамалов Олександр Олегович, група РІ-11

1. Мета роботи

Навчитися використовувати функції, що містять розгалуження та цикли з рекурентними співвідношеннями.

2. Умова завдання

Варіант 7.

Обчислити і вивести на екран у вигляді таблиці значення виразу від дійсного z на інтервалі від $z_{поч}$ до $z_{кін}$ з n відрізками розбиття інтервалу. Параметри Zпоч, Zкін, n вводяться з клавіатури. Таблиця має містити заголовок та шапку. Кожний рядок таблиці має містити значення z та значення виразу. Використовувати допоміжний алгоритм, реалізований за допомогою окремої функції. При обчисленні значення доданків в сумі використовувати рекурентні співвідношення.

$$k(z^2+1)-k(z^2-1)+2k(z) \quad , \quad де$$

$$k(x)=\begin{cases}\dfrac{e^x+\sin x}{\cos^2 x+1} & , \quad |x|\ge 1\\ \dfrac{1}{e^x}\displaystyle\sum_{i=0}^{5}\frac{x^i}{i!} & , \quad |x|<1\end{cases}$$

Рисунок 1 – Умова завдання 5.3, варіант 7

3. Математична модель

Для $|x| < 1$ доданки суми $a_i = x^i / i!$, $i = 0, \ldots, 5$. Рекурентне співвідношення: $a_0 = 1$; $a_i = a_{i-1} \cdot x / i$, $i = 1, \ldots, 5$ ($R_i = x / i$). Результат гілки: $k(x) = (\Sigma a_i) / e^x$.

Для $|x| \ge 1$: $k(x) = (e^x + \sin x) / (\cos^2 x + 1)$.

Крок таблиці $\Delta z = (z_{кін} - z_{поч}) / n$; кількість рядків $n + 1$. Вхідні дані: $z_{поч} \le z_{кін}$, $n > 0$, межі скінченні. Тригонометричні функції використовують радіани.

4. Структура програми

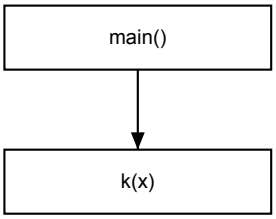


Рисунок 2 – Структурна схема програми (ієрархія викликів)

Таблиця 1 – Специфікація функцій програми

Функція	Призначення	Вхідні параметри	Результат	Викликає
int main()	введення меж і n; виведення таблиці F(z)	–	код завершення	k – тричі на рядок
double k(double x)	допоміжна функція з розгалуженням і рекурентною сумою	x – дійсне число	значення k(x)	exp, sin, cos

Таблиця 2 – Змінні програми

Ім'я	Тип	Функція	Призначення
zp, zk	double	main	початок і кінець інтервалу
n	int	main	кількість відрізків розбиття
dz	double	main	крок таблиці
i	int	main / k	номер рядка таблиці / номер доданка
z	double	main	поточне значення аргументу
y	double	main	значення F(z)
x	double	k	параметр функції
a	double	k	поточний доданок суми
sum	double	k	сума доданків

5. Алгоритми програми

Блок-схеми виконано за ГОСТ 19.701-90 (ДСТУ ISO 5807:2016): «Початок» і «Кінець» – символ «термінатор», введення/виведення – «дані» (паралелограм), обчислення – «процес», виклик функції – «зумовлений процес», умова – «рішення». Лінії потоку, що йдуть зверху вниз і зліва направо, стрілками не позначено; стрілку показано лише на лінії повернення (повторення циклу).

UML-діаграми діяльності (UML 2.5) виконано в рамці «act»: початковий вузол – зафарбоване коло, кінцевий – коло з точкою, дії – прямокутники із заокругленими кутами (дія-виклик позначена символом «вила»), розгалуження та злиття – ромби, умови переходів – сторожові вирази у квадратних дужках. Усі переходи позначено стрілками.

5.1. Головна функція main()

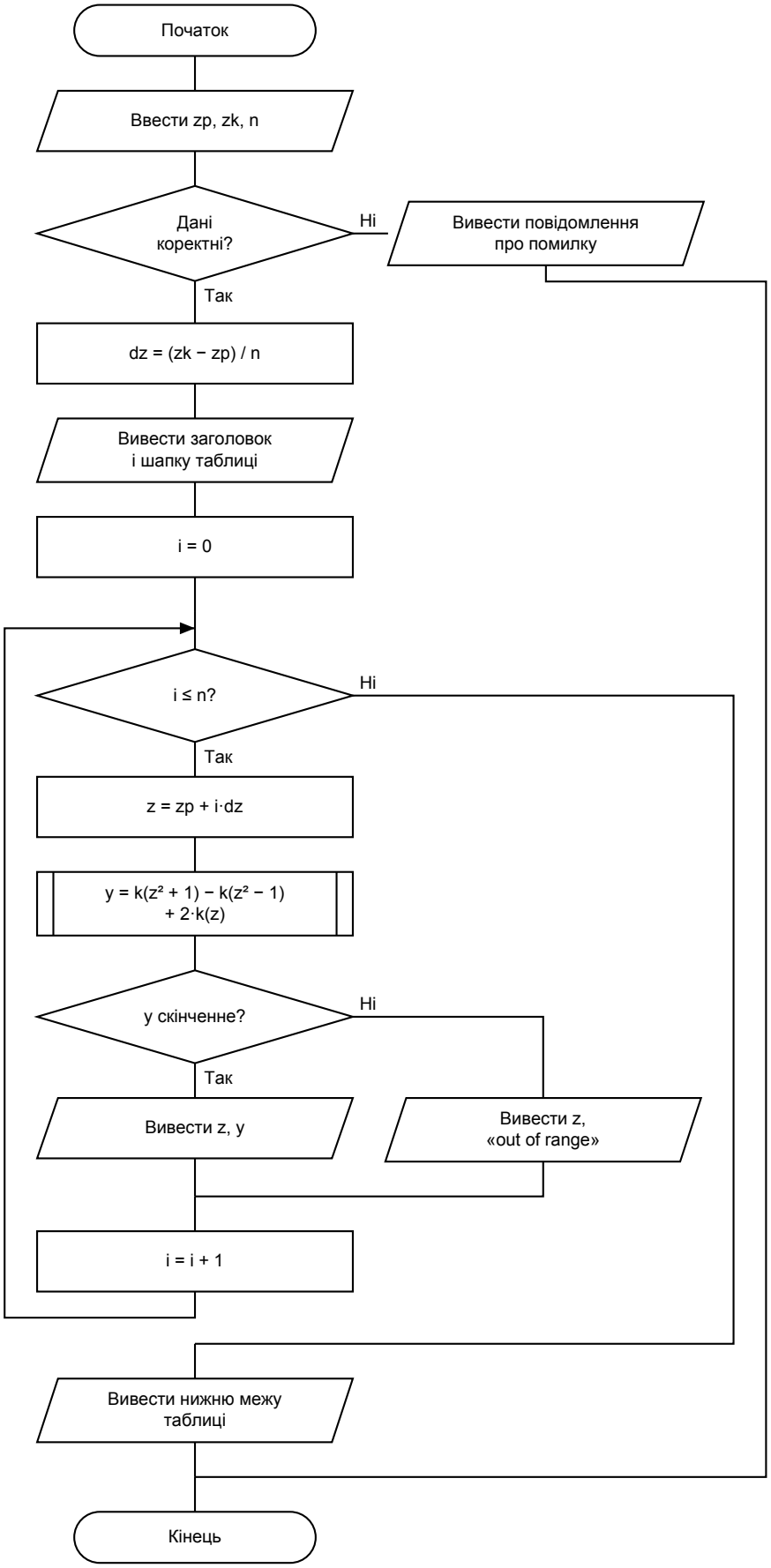


Рисунок 3 – Блок-схема функції main()

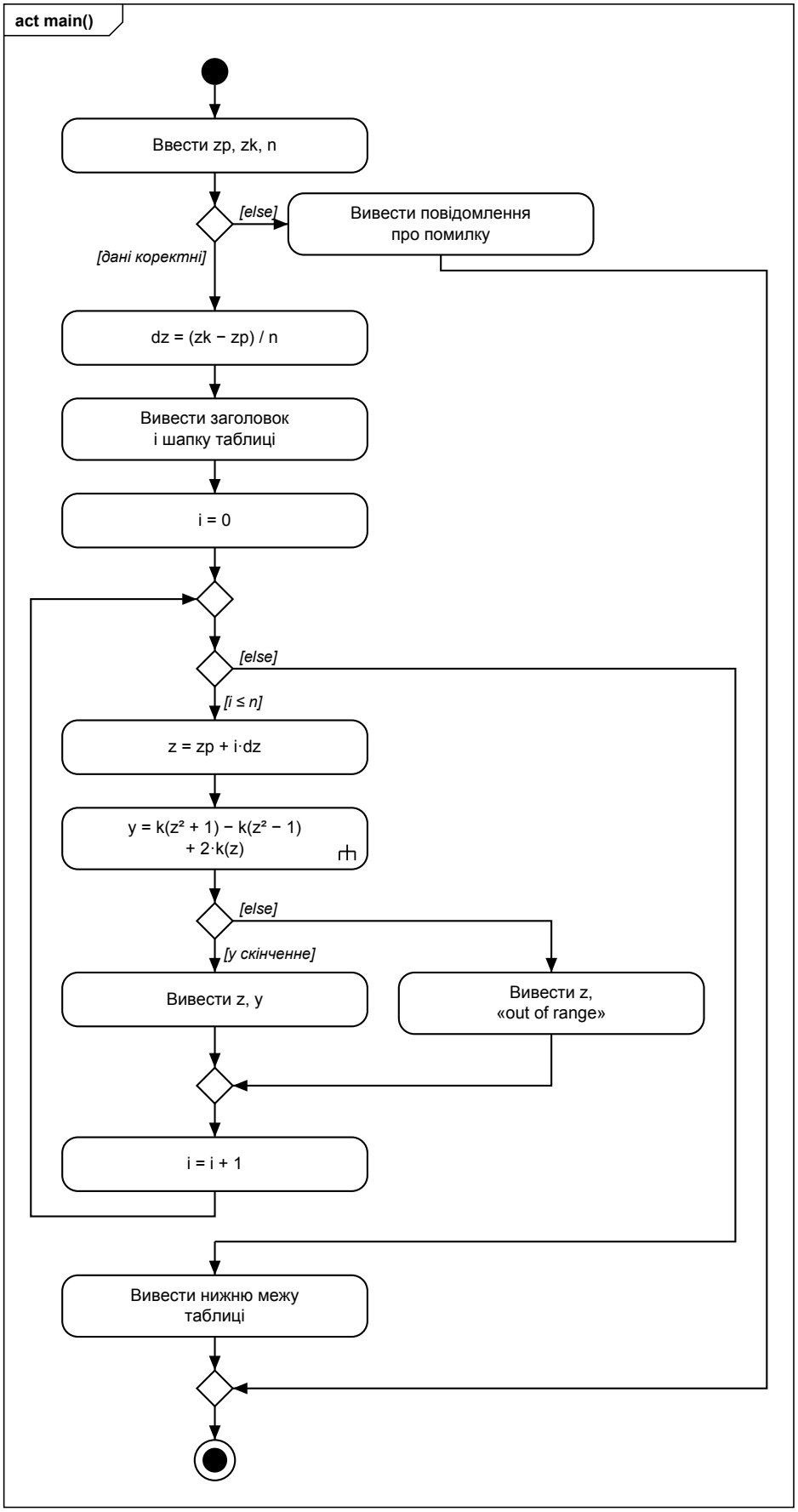


Рисунок 4 – UML-діаграма діяльності функції main()

5.2. Допоміжна функція k(x)

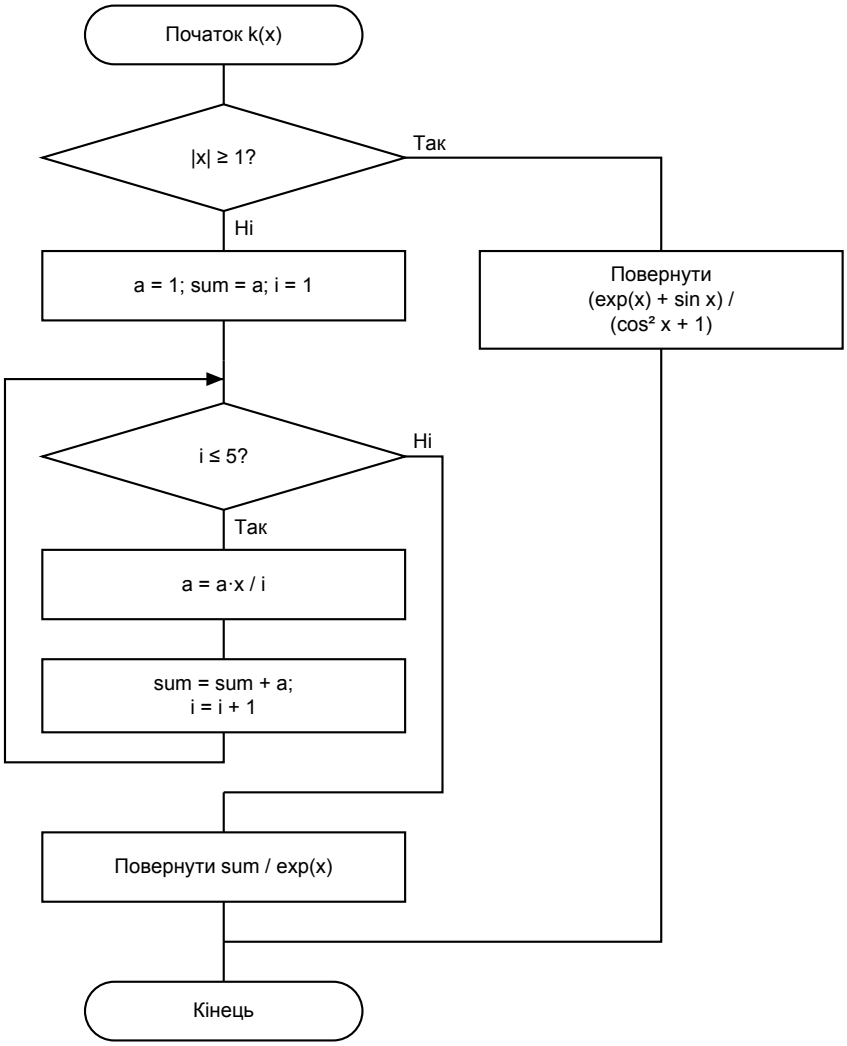


Рисунок 5 – Блок-схема функції k(x)

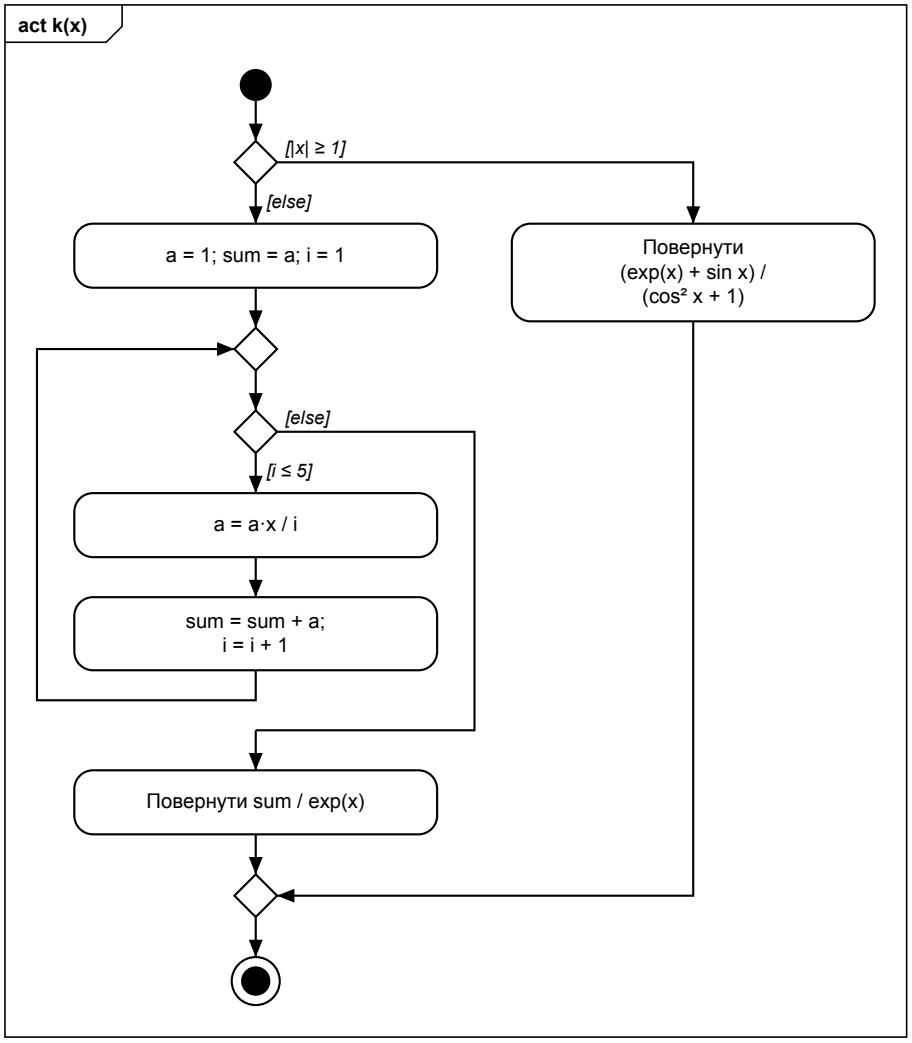


Рисунок 6 – UML-діаграма діяльності функції k(x)

6. Текст програми

Кожна дія програми супроводжується коментарем. Головну функцію main() вимкнено директивою #ifndef UNIT_TEST, щоб файл можна було підключити до тестів.

Файл `lab_5_3.cpp`

```
// Лабораторна робота 5.3, варіант 7. Джамалов О. О., PI-11.
// F(z) = k(z^2 + 1) - k(z^2 - 1) + 2k(z) на [z_begin, z_end] з n відрізками.
#include <cmath>    // abs, exp, sin, cos, isfinite
#include <iomanip>  // setw, setprecision, fixed
#include <iostream> // cin, cout, cerr
using namespace std;

double k(double x); // оголошення допоміжної функції

#ifndef UNIT_TEST
int main()
{
    double zp, zk;                          // межі інтервалу
    int n;                                  // кількість відрізків розбиття
    cout << "z_begin z_end n: ";            // запрошення до введення
    if (!(cin >> zp >> zk >> n) || !isfinite(zp) || !isfinite(zk) ||
        !isfinite(zk - zp) || zp > zk || n <= 0) // перевірка введення
    {
        cerr << "Error: z_begin <= z_end, n > 0.\n"; // повідомлення про помилку
        return 1;                           // завершення з кодом помилки
    }

    const double dz = (zk - zp) / n;        // крок таблиці
    const char* line = "+----------------+----------------------+\n"; // межа таблиці
    cout << "Lab 5.3 / Variant 7\n"         // заголовок
         << "F(z) = k(z*z+1) - k(z*z-1) + 2*k(z)\n" << line;
    cout << "|        z       |         F(z)         |\n" << line; // шапка таблиці
    cout << fixed << setprecision(8);       // формат чисел
    for (int i = 0; i <= n; ++i)            // n відрізків дають n + 1 точку
    {
        const double z = zp + i * dz;       // поточне значення z
        const double y = k(z * z + 1) - k(z * z - 1) + 2 * k(z); // значення F(z)
        if (isfinite(y))                    // чи результат скінченний?
            cout << "| " << setw(15) << z << " | " << setw(20) << y << " |\n";
        else                                // переповнення типу double
            cout << "| " << setw(15) << z << " | " << setw(20) << "out of range" << " |\n";
    }
    cout << line;                           // нижня межа таблиці
    return 0;                               // успішне завершення
}
#endif

// Допоміжна функція k(x):
// (e^x + sin x) / (cos^2 x + 1), якщо |x| >= 1;
// (1 / e^x) * sum_{i=0..5} x^i / i!, якщо |x| < 1.
double k(double x)
{
    if (abs(x) >= 1)                        // перша гілка
        return (exp(x) + sin(x)) / (cos(x) * cos(x) + 1);

    double a = 1;                           // a_0 = x^0 / 0! = 1
    double sum = a;                         // сума починається з a_0
    for (int i = 1; i <= 5; ++i)            // доданки a_1 ... a_5
    {
        a *= x / i;                         // a_i = a_(i-1) * x / i
        sum += a;                           // додаємо доданок до суми
    }
    return sum / exp(x);                    // ділимо суму на e^x
}
```

7. Автоматизоване тестування

Файл `test_5_3.cpp`

```
// Unit-тести функції k із завдання 5.3.
#include <algorithm>    // max
#include <cmath>        // pow, exp, sin, cos
#include <iomanip>      // setw, setprecision
#include <iostream>     // cout

#define UNIT_TEST       // вимикаємо main() у файлі програми
#include "lab_5_3.cpp"  // підключаємо функцію k

int main()
{
    // Незалежний еталон: явний многочлен, без рекурентної формули.
    const double xs[] = {-2, -1, -0.999999, -0.5, 0,
                         0.5, 0.999999, 1, 2};   // обидві гілки та межі
    int failed = 0;                              // лічильник невдалих тестів
    for (double x : xs)                          // перебір аргументів
    {
        double expected;                         // очікуване значення
        if (std::abs(x) >= 1)                    // перша гілка
            expected = (std::exp(x) + std::sin(x)) /
                       (1 + std::pow(std::cos(x), 2));
        else                                     // друга гілка
            expected = (1 + x + std::pow(x, 2) / 2 +
                        std::pow(x, 3) / 6 + std::pow(x, 4) / 24 +
                        std::pow(x, 5) / 120) / std::exp(x);

        const double actual = k(x);              // фактичне значення
        const bool ok = std::isfinite(actual) && // порівняння з допуском
            std::abs(actual - expected) <=
            1e-12 * std::max(1.0, std::abs(expected));
        std::cout << (ok ? "PASS" : "FAIL")
                  << " x=" << std::setw(10) << x
                  << " k(x)=" << std::setprecision(12)
                  << actual << '\n';             // результат тесту
        if (!ok)
            ++failed;                            // рахуємо помилку
    }
    std::cout << "Tests: 9, passed: " << 9 - failed
              << ", failed: " << failed << '\n'; // підсумок
    return failed == 0 ? 0 : 1;                  // код завершення для CTest
}
```

Фактичний вивід тестової програми:

```
PASS x=        -2 k(x)=-0.659714057482
PASS x=        -1 k(x)=-0.366577753206
PASS x= -0.999999 k(x)=0.996703359754
PASS x=      -0.5 k(x)=0.99996662486
PASS x=         0 k(x)=1
PASS x=       0.5 k(x)=0.999985835063
PASS x=  0.999999 k(x)=0.999405818248
PASS x=         1 k(x)=2.7553832111
PASS x=         2 k(x)=7.0733956696
Tests: 9, passed: 9, failed: 0
```

Таблиця 3 – Результати unit-тестів

№	x	Гілка	Еталон	Результат
1	−2	$\|x\| \geq 1$	пряма формула	пройдено
2	−1	$\|x\| \geq 1$ (межа)	пряма формула	пройдено
3	−0,999999	$\|x\| < 1$ (біля межі)	явний многочлен	пройдено
4	−0,5	$\|x\| < 1$	явний многочлен	пройдено
5	0	$\|x\| < 1$	явний многочлен	пройдено
6	0,5	$\|x\| < 1$	явний многочлен	пройдено
7	0,999999	$\|x\| < 1$ (біля межі)	явний многочлен	пройдено
8	1	$\|x\| \geq 1$ (межа)	пряма формула	пройдено
9	2	$\|x\| \geq 1$	пряма формула	пройдено

8. Результати виконання програми

Компілятор clang++, стандарт C++17, ключі -Wall -Wextra -Wpedantic (попереджень немає). Вхідні дані: `-1 1 4`.

```
z_begin z_end n: -1 1 4
Lab 5.3 / Variant 7
F(z) = k(z*z+1) - k(z*z-1) + 2*k(z)
+-----------------+----------------------+
|        z        |         F(z)         |
+-----------------+----------------------+
|     -1.00000000 |           5.34024016 |
|     -0.50000000 |           5.03825666 |
|      0.00000000 |           5.12196096 |
|      0.50000000 |           5.03829508 |
|      1.00000000 |          11.58416209 |
+-----------------+----------------------+
```

9. Посилання на Git-репозиторій

https://github.com/dzhamsasha55-dev/lab-5

Висновки

Реалізовано допоміжну функцію k(x) з розгалуженням за |x| і циклом обчислення суми. Рекурентна формула $a_i = a_{i-1} \cdot x / i$ дозволяє отримувати доданки без окремого обчислення степенів і факторіалів.

Основна програма виводить n + 1 значень виразу у вигляді таблиці з рамкою, включно з обома кінцями інтервалу; при переповненні типу double виводиться «out of range». Усі 9 unit-тестів пройдено (еталон обчислено незалежно, без рекурентності).